LAPORAN PRAKTIKUM MICROPROCESSOR & INTERFACE

PROGRAM PENJUMLAHAN OKTAL

PRAKTIKAN

Nama : Elsadanika Maura Sitorus

NIM : 211344013

PROGRAM STUDI D4 TEKNIK TELEKOMUNIKASI

JURUSAN TEKNIK ELEKTRO

POLITEKNIK NEGERI BANDUNG

2022/2023

KETERANGAN

1. Kelompok : -
2. Judul Laporan : Program Penjumlahan Oktal
3. Tanggal Praktikum : 07 Februari 2023
4. Tanggal Pengumpulan : 12 Februari 2023
5. Nama Praktikan : Elsadanika Maura Sitorus
6. Nama Partner : -
7. Nama Dosen Pembimbing : Ferry Satria, BSEE., M. T.
   Rahmawati Hasanah, S. ST., M. T.

DAFTAR ISI

KETERANGAN i
DAFTAR ISI ii

A. Tujuan Praktikum 1
B. Dasar Teori 1
C. Alat dan Komponen yang Digunakan 3
D. Langkah Kerja 3
E. Data Pengamatan 4
F. Analisis Data 20
G. Kesimpulan 26
H. Daftar Pustaka 26

A. Tujuan Praktikum

1. Memahami bahasa Assembly
2. Memahami cara penggunaan simulator EMU 8086
3. Memahami pemanfaatan EMU 8086 simulator untuk menjumlahkan data bilangan oktal menggunakan bahasa pemrograman Assembly.

B. Dasar Teori

1. Perintah Looping, Memory, dan Data Area.

Arahan assembler define digunakan sebagai alokasi ruang penyimpanan dan untuk menginisialisasi satu atau lebih byte.

a) Mengalokasikan Ruang Penyimpanan untuk Data yang Diinisialisasi

Sintaks untuk pernyataan alokasi penyimpanan untuk data yang diinisialisasi adalah sebagai berikut:

```
[nama-variabel] define-directive initial-value [,initial-value]...
```

Nama variabel adalah pengidentifikasi untuk setiap ruang penyimpanan. Assembler mengaitkan nilai offset untuk setiap nama variabel yang ditentukan dalam segmen data. Ada lima bentuk dasar dari direktif define yang ditampilkan pada tabel berikut:

Perintah	Keterangan	Storage Space
DB	Define Byte	1 byte
DW	Define Word	2 bytes
DD	Define Doubleword	4 bytes
DQ	Define Quadword	8 bytes
DT	Define Ten Bytes	10 bytes

b) Perintah Looping

Instruksi untuk transfer perintah selama eksekusi disesuaikan dengan kondisinya, berikut ini adalah instruksi untuk proses looping dalam Bahasa assembly pada 8086:

Instructions	Conditions
JA/JNBE	Jump if above/not below/equal instruction satisfies
JAE/JNB	Jump if above/not below
JBE/JNA	Jump if below/equal/ not above

JC	Jump if carry flag CF = 1
JE/JZ	Jump if equal/zero flag ZF = 1
JG/JNLE	Jump if greater/not less than/equal
JGE/JNL	Jump if greater than/equal/not less
JL/JNGE	Jump if less than/not greater than/equal
JLE/JNG	Jump if less than/equal/if not greater
JNC	Jump if no carry flag (CF = 0)
JNE/JNZ	Jump if not equal/zero flag ZF = 0
JNO	Jump if no overflow flag OF = 0
JNP/JPO	Jump if not parity/parity odd PF = 0
JNS	Jump if not sign SF = 0
JO	Jump if overflow flag OF = 1
JP/JPE	Jump if parity/parity even PF = 1
JS	Jump if sign flag SF = 1

2. Penjumlahan Bilangan Oktal

Bilangan Oktal = $8^3\ 8^2\ 8^1\ 8^0$

Cara konversi decimal ke oktal adalah sebagai berikut.

	$4327_{(8)}$ → …(10)
$7 . 8^0$ =	$7 . 1$ → 7
$2 . 8^1$ =	$2 . 8$ → 16
$3 . 8^2$ =	$3 . 64$ → 192
$4 . 8^3$ =	$4 . 512$ → 2048
---------- +	---------- +
	$4327_{(8)}$ = $2263_{(10)}$

Cara konversi heksadesimal ke oktal adalah dengan menambahkan 8 jika uji satuan bernilai >7. Contohnya adalah sebagai berikut.

77	
46	
------- +	
BD	Nilai D > 7
8 8	
------- +	
145	

Bilangan oktal memiliki satuan, delapanan, enam puluh empatan, dan selanjutnya. Untuk memisahkan satuan dan delapan, digunakan perintah logic AND dengan

symbol Λ. Contoh untuk memisahkan BD antara satuan dengan delapanannya adalah sebagai berikut.

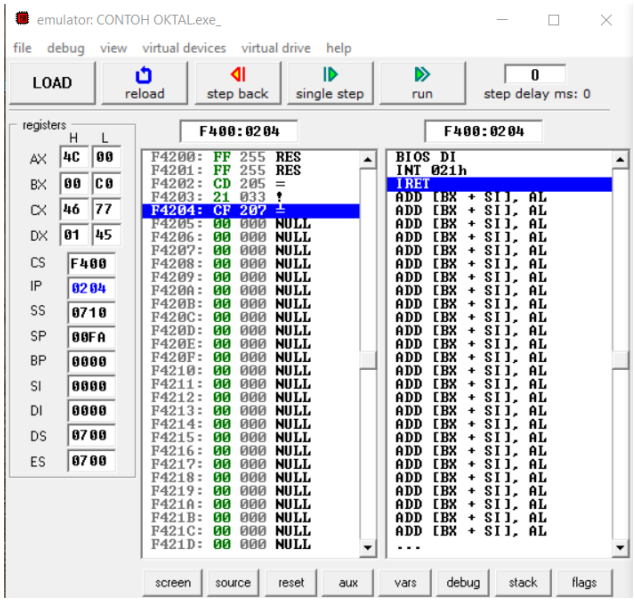
1011 1101
0000 1111 Λ → AND dengan 00FH
---------- +
0000 1101 = 0D → dapat dilakukan proses CMP 07H

1011 1101
1111 0000 Λ → AND dengan 0F0H
---------- +
1011 0000 = B0 → dapat dilakukan proses CMP 70H

Berikut ini contoh program menjumlahkan 77+46 (heksadesimal) = 145 (oktal) dengan hasil oktal yang disimpan pada register DX.

original source code

```
01  TITLE PENJ OKTAL
02  .MODEL SMALL
03  .STACK 100H
04  .DATA
05  .CODE
06  MAIN PROC
07  MOV DX,0
08  MOV DL,CL
09  ADD DL,CH
10  MOV BL,DL
11  AND BL,0FH   ;SAAT DIUJI
12  CMP BL,7
13  JBE L1       ;SATUAN SUDAH SELESAI DIUJI
14
15  ADD DL,8     ;KOREKSI
16  L1:  MOV BL,DL    ;HARUS DIPINDAH SUPAYA DATA DI DL GA RUSAK/HILANG
17  AND BL,0F0H
18  CMP BL,70H
19  JBE L2
20  ADD DX,80H   ;KOREKSI
21  ;ADC DH,0
22
23  L2:  MOV AH,4CH
24  INT 21H
25  MAIN ENDP
26  END MAIN
```



C. Alat dan Komponen yang Digunakan

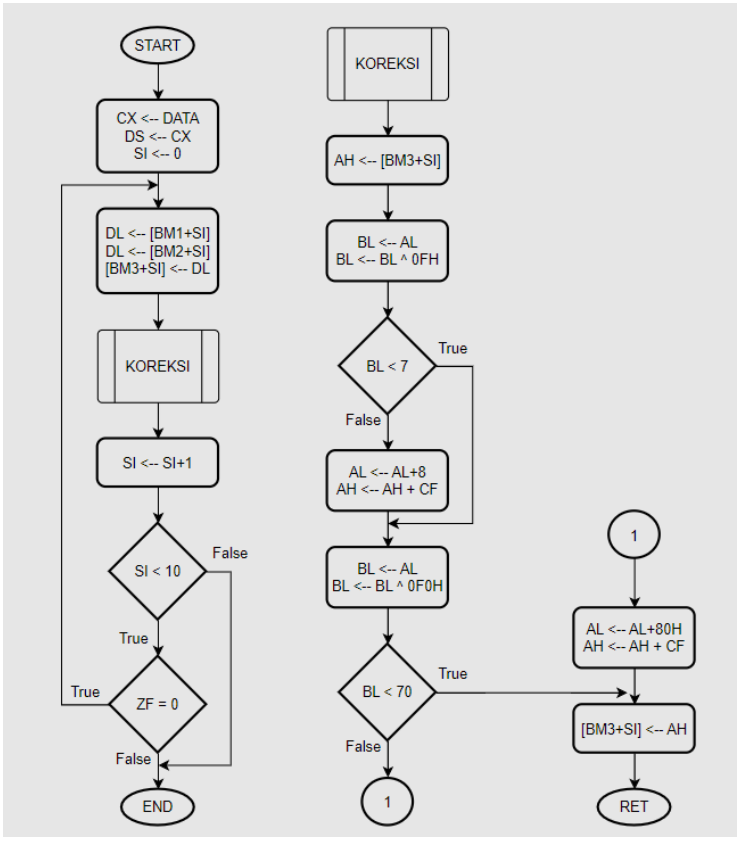
- Laptop/Personal Computer
- Simulator EMU 8086

D. Langkah Kerja

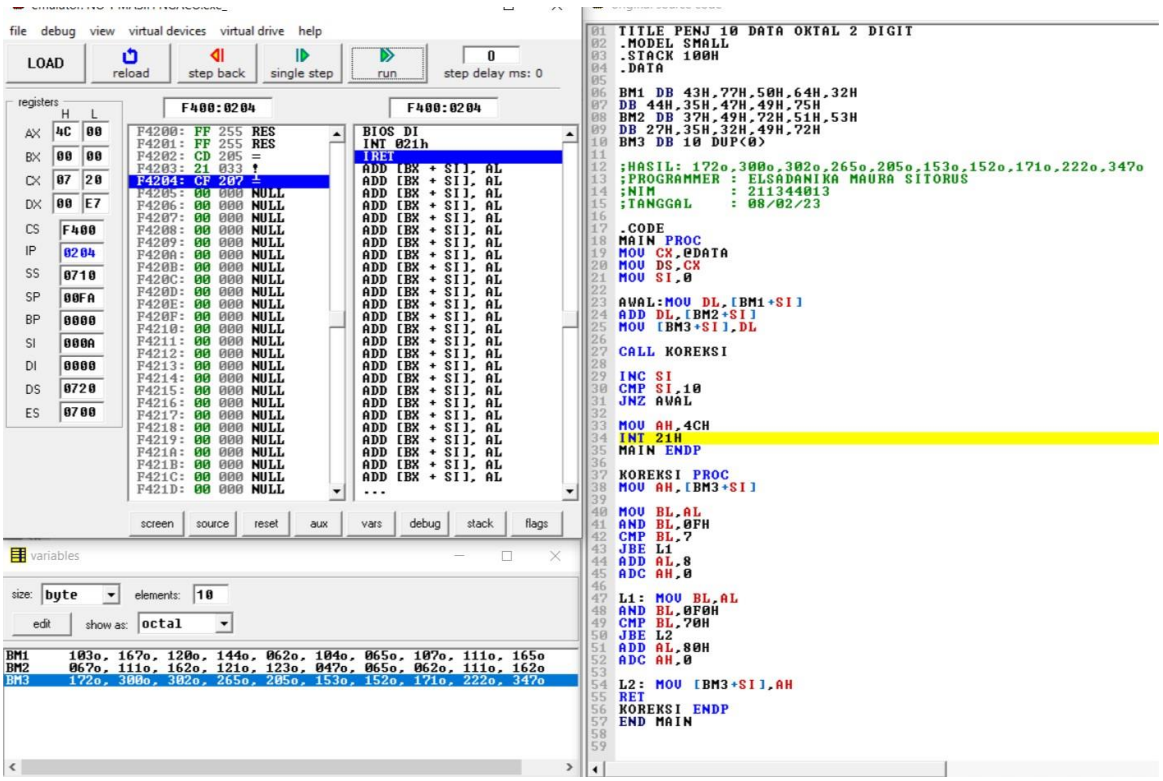
1. Buat diagram alir dan program untuk menjumlahkan 10 pasang data oktal 2 digit yang berada pada BM1 dan BM2. Simpan hasil penjumlahan pada BM3.
   BM1 DB 43H,77H,50H,64H,32H,44H,35H,47H,49H,75H
   BM2 DB 37H,49H,72H,51H,53H,27H,35H,23H,49H,72H
   BM3 DB 10 DUP(0)
2. Buat diagram alir dan program untuk menjumlahkan 10 pasang data oktal 4 digit yang berada pada BM1 dan BM2. Simpan hasil penjumlahan pada BM3 dan BM4.
   BM1 DW 1663H,7347H,6351H,4324H,5632H,4423H,1557H,4700H,4602H,7535H
   BM2 DW 3437H,4346H,3463H,3472H,4443H,2762H,4023H,6723H,3225H,6251H
   BM3 DW 10 DUP(0)
   BM4 DB 10 DUP(0)
3. Buat diagram alir dan program untuk menjumlahkan data oktal 7 digit yang berada pada BM1 dan BM2. Simpan hasil penjumlahan pada BM3.
   BM1 7403456H,2674321H,7654321H,2354765H,2211775H
   BM2 6234567H,6766550H,5566772H,7234567H,6217755H
   BM3 0,0,0,0,0
4. Buat diagram alir dan program untuk menjumlahkan 4 pasang data oktal 11 digit. Data input dan outputnya tentukan sendiri.
5. Buat program untuk menjumlahkan 9 data oktal berikut: 7654324 + 6767543 + 7766554 + 7454326 + 6426765 + 7755643 + 7776665 + 5556667 + 7747576. Simpan data input dan output pada data area.
6. Buat program untuk menjumlahkan 9 data oktal berikut: 74324 + 67543 + 77554 + 74326 + 64265 + 77643 + 76765 + 67677 + 74743. Simpan data input dan output pada data area.

E. Data Pengamatan

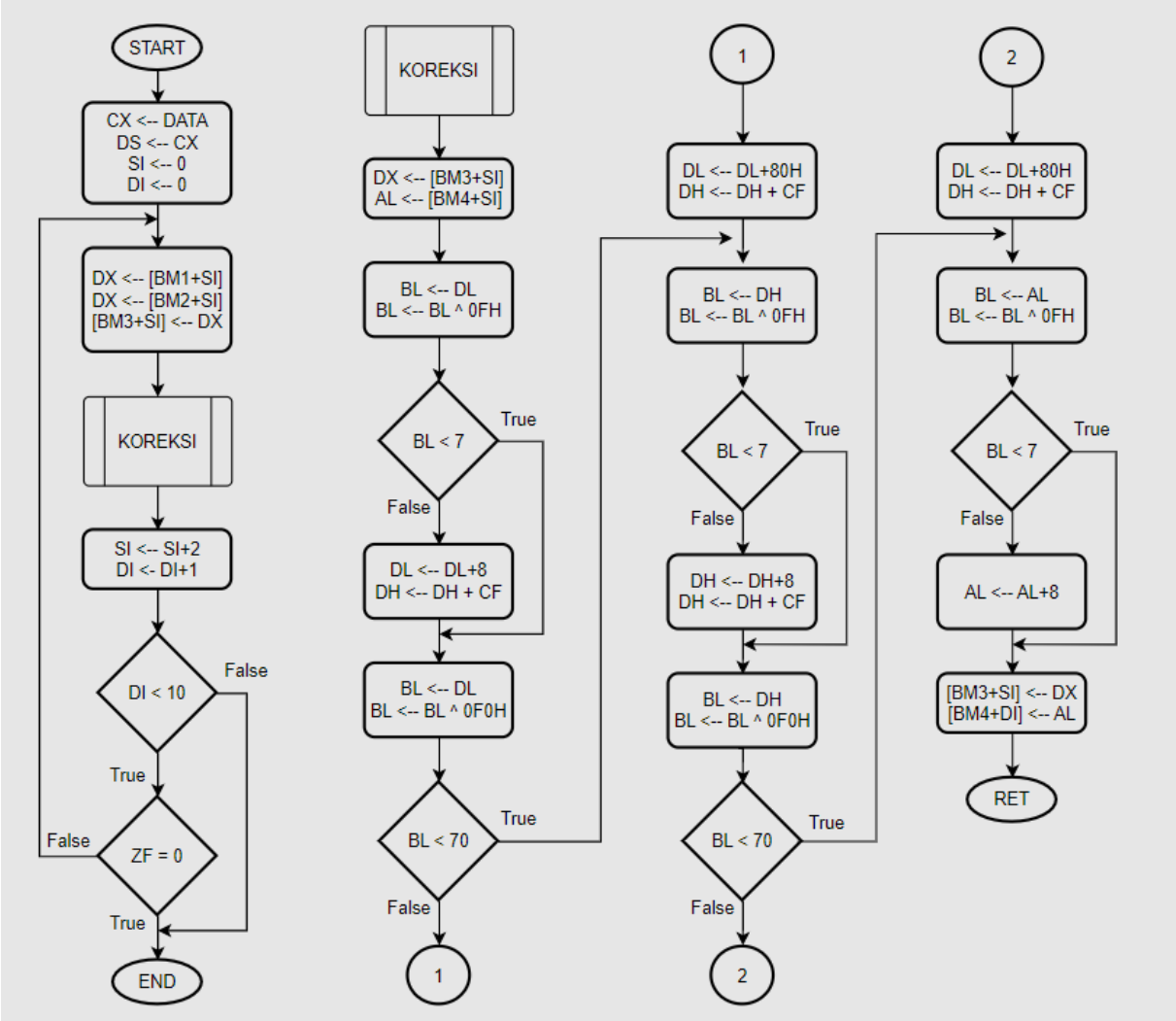
1. Flowchart



Program dan Output



2. Flowchart



Program

```
TITLE PENJ 10 PASANG DATA OKTAL 4 DIGIT ;SIMPAN DI BM3 DAN BM4
.MODEL SMALL
.STACK 100H
.DATA
BM1 DW 1663H,7347H,6351H,4324H
    DW 5632H,4423H,1557H,4700H
    DW 4602H,7535H
BM2 DW 3437H,4346H,3463H,3472H
    DW 4443H,2762H,4023H,6723H
    DW 3225H,6251H
BM3 DW 10 DUP(0)
BM4 DB 10 DUP(0)

;HASIL BM3 = 5322h,3715h,2034h,0016h,2275h,7405h,5602h,3623h,0027h,
6006h dan BM4 = 00h, 01h, 01h, 01h, 01h, 00h, 00h, 01h, 01h, 01h
;PROGRAMMER : ELSADANIKA MAURA SITORUS
;NIM        : 211344013 - 2NK
;TANGGAL    : 08/02/23
```

```
.CODE
MAIN PROC
    MOV CX,@DATA
    MOV DS,CX
    MOV SI,0
    MOV DI,0
    MOV DX,0

AWAL:MOV DX,[BM1+SI]
     ADD DX,[BM2+SI]
     MOV [BM3+SI],DX

     CALL KOREKSI

     INC SI
     INC SI
     INC DI
     CMP DI,10
     JNZ AWAL

    MOV AH,4CH
    INT 21H
    MAIN ENDP

KOREKSI PROC
    MOV DX,[BM3+SI]
    MOV AL,[BM4+DI]

     MOV BL,DL
     AND BL,0FH
     CMP BL,7
     JBE L1
     ADD DL,8
     ADC DH,0

L1: MOV BL,DL
    AND BL,0F0H
    CMP BL,70H
    JBE L2
    ADD DL,80H
    ADC DH,0

L2: MOV BH,DH
    AND BH,0FH
    CMP BH,7
    JBE L3
    ADD DH,8
    ADC DH,0

L3: MOV BH,DH
    AND BH,0F0H
    CMP BH,70H
    JBE L4
    ADD DH,80H
    ADC AL,0

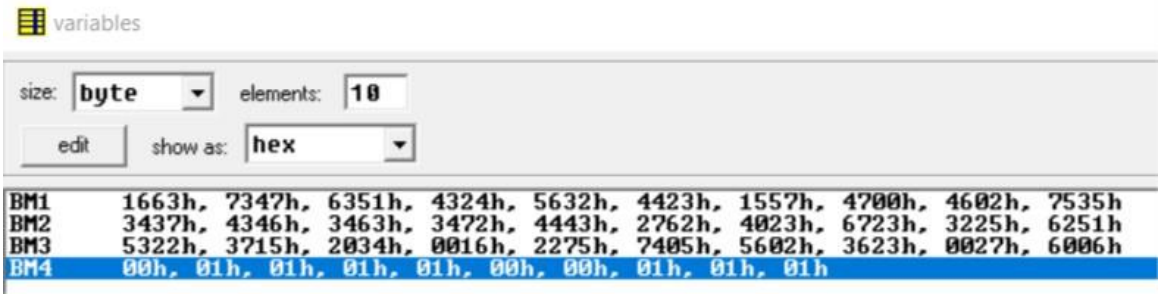
L4: MOV BL,AL
    AND BL,0FH
```

```
    CMP BL,7
    JBE L5
    ADD AL,8

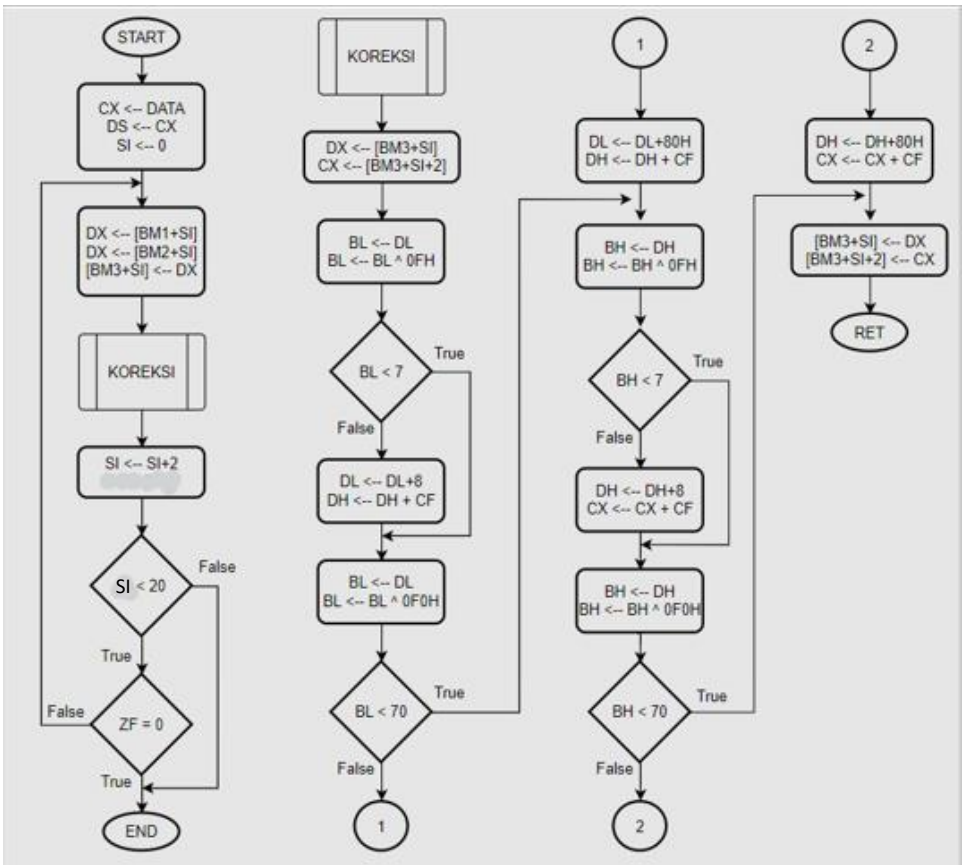
L5: MOV [BM3+SI],DX
    MOV [BM4+DI],AL

    RET
    KOREKSI ENDP
END MAIN
```

Output



3. Flowchart



Program

```
TITLE PENJ DATA OKTAL 7 DIGIT
.MODEL SMALL
.STACK 100H
.DATA
;BM1 7403456H,2674321H,7654321H,2354765H,2211775H
;BM2 6234567H,6766550H,5566772H,7234567H,6217755H
;HASIL : 15640245o,11663071o,15443313o,11611554o,10431752o

BM1 DW 3456H,0740H,4321H,0267H
    DW 4321H,0765H,4765H,0235H
    DW 1775H,0221H
BM2 DW 4567H,0623H,6550H,0676H
    DW 6772H,0556H,4567H,0723H
    DW 7755H,0621H
BM3 DW 10 DUP(0)

;PROGRAMMER : ELSADANIKA MAURA SITORUS
;NIM        : 211344013
;TANGGAL    : 08/02/23

.CODE
MAIN PROC
    MOV CX,@DATA
    MOV DS,CX
    MOV SI,0

AWAL:MOV DX,[BM3+SI]
     ADD DX,[BM1+SI]
     ADD DX,[BM2+SI]
     MOV [BM3+SI],DX

     CALL KOREKSI

     INC SI
     INC SI
     CMP SI,20
     JNZ AWAL

    MOV AH,4CH
    INT 21H
    MAIN ENDP

KOREKSI PROC
    MOV DX,[BM3+SI]
    MOV CX,[BM3+SI+2]

     MOV BL,DL
     AND BL,0FH
     CMP BL,7
     JBE L1
     ADD DL,8
     ADC DH,0

L1: MOV BL,DL
    AND BL,0F0H
    CMP BL,70H
    JBE L2
```

```
     ADD DL,80H
     ADC DH,0

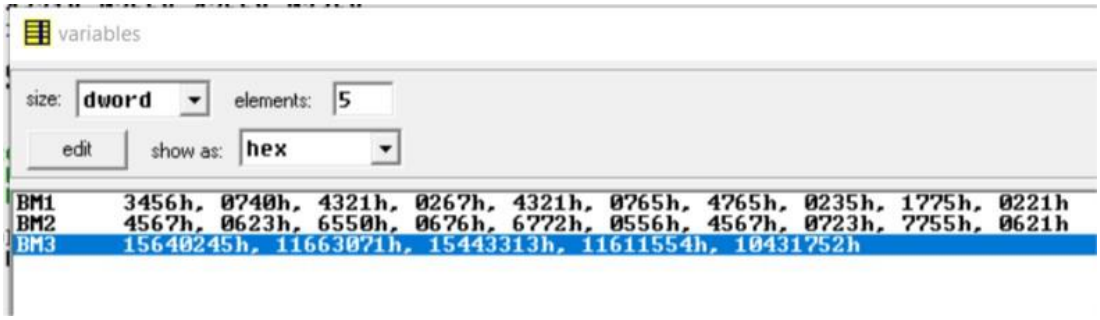
L2:  MOV BH,DH
     AND BH,0FH
     CMP BH,7
     JBE L3
     ADD DH,8
     ADC CX,0

L3:  MOV BH,DH
     AND BH,0F0H
     CMP BH,70H
     JBE L4
     ADD DH,80H
     ADC CX,0

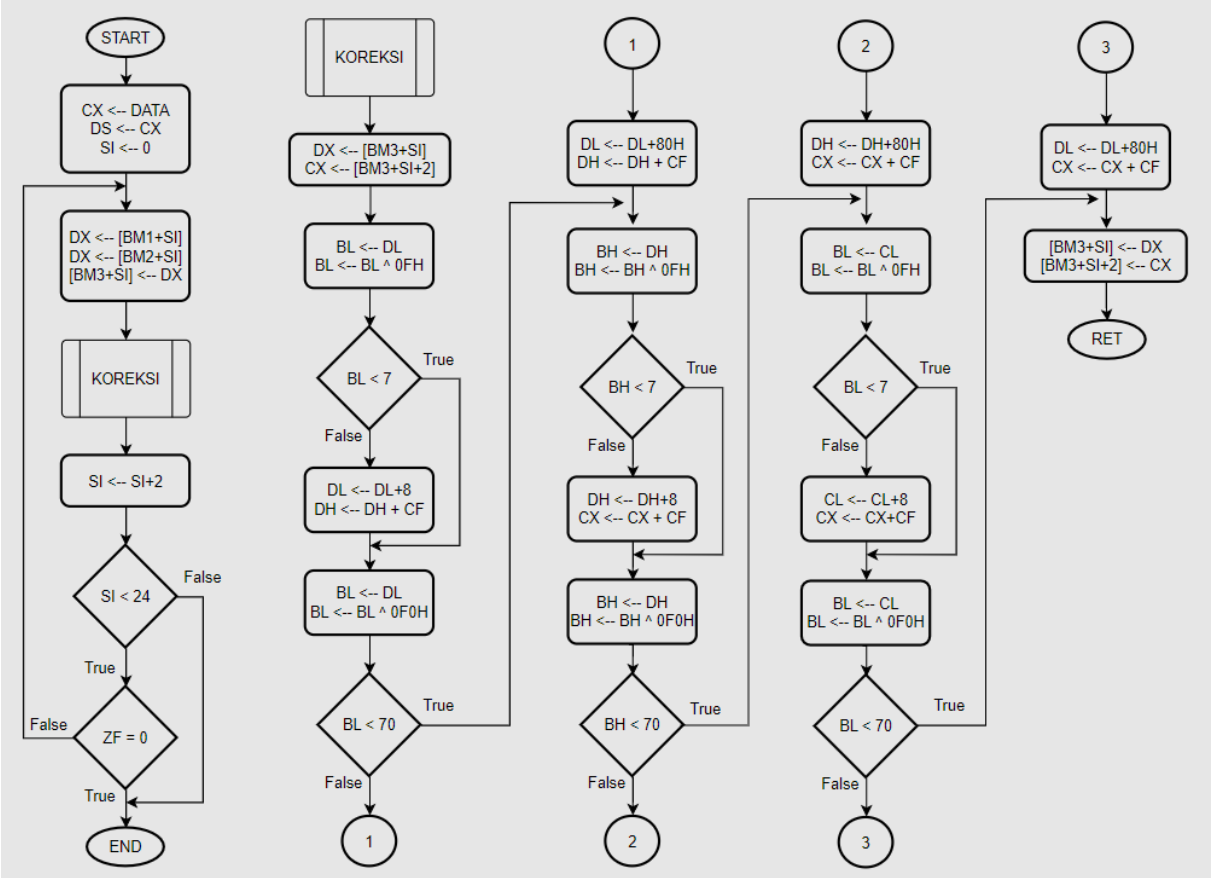
L4:  MOV [BM3+SI],DX
     MOV [BM3+SI+2],CX

     RET
     KOREKSI ENDP
END MAIN
```

Output



4. Flowchart



Program

```
TITLE PENJ 4 PASANG DATA OKTAL 11 DIGIT
.MODEL SMALL
.STACK 100H
.DATA

;DATA : 11122233344 (DUP (4))

BM1 DW 3344H,2223H,0111H,3344H
    DW 2223H,0111H,3344H,2223H
    DW 0111H,3344H,2223H,0111H
BM2 DW 3344H,2223H,0111H,3344H
    DW 2223H,0111H,3344H,2223H
    DW 0111H,3344H,2223H,0111H

BM3 DW 8 DUP(0)

;HASIL : 22244466710o (DUP (4))

.CODE
MAIN PROC
    MOV CX,@DATA
    MOV DS,CX
    MOV SI,0
```

```
AWAL:MOV DX,[BM3+SI]
     ADD DX,[BM1+SI]
     ADD DX,[BM2+SI]
     MOV [BM3+SI],DX

     CALL KOREKSI

     INC SI
     INC SI
     CMP SI,24
     JNZ AWAL

    MOV AH,4CH
    INT 21H
    MAIN ENDP

KOREKSI PROC
    MOV DX,[BM3+SI]
    MOV CX,[BM3+SI+2]

     MOV BL,DL
     AND BL,0FH
     CMP BL,7
     JBE L1
     ADD DL,8
     ADC DH,0

L1: MOV BL,DL
    AND BL,0F0H
    CMP BL,70H
    JBE L2
    ADD DL,80H
    ADC DH,0

L2: MOV BH,DH
    AND BH,0FH
    CMP BH,7
    JBE L3
    ADD DH,8
    ADC CX,0

L3: MOV BH,DH
    AND BH,0F0H
    CMP BH,70H
    JBE L4
    ADD DH,80H
    ADC CX,0

L4:  MOV BL,CL
     AND BL,0FH
     CMP BL,7
     JBE L5
     ADD DL,8
     ADC CX,0

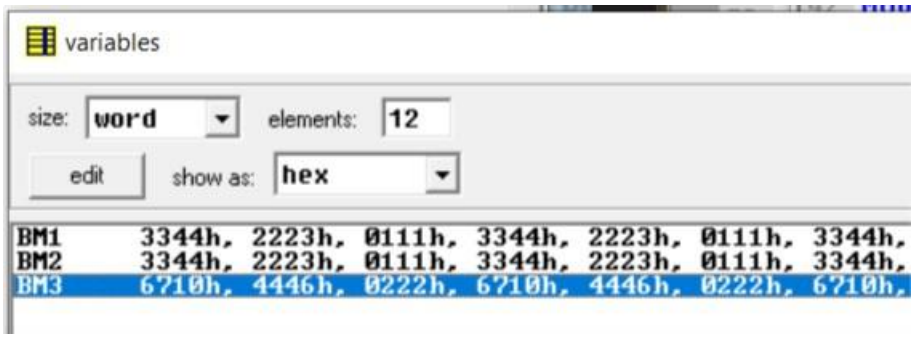
L5: MOV BL,CL
    AND BL,0F0H
    CMP BL,70H
    JBE L6
```

```
    ADD DL,80H
    ADC CX,0

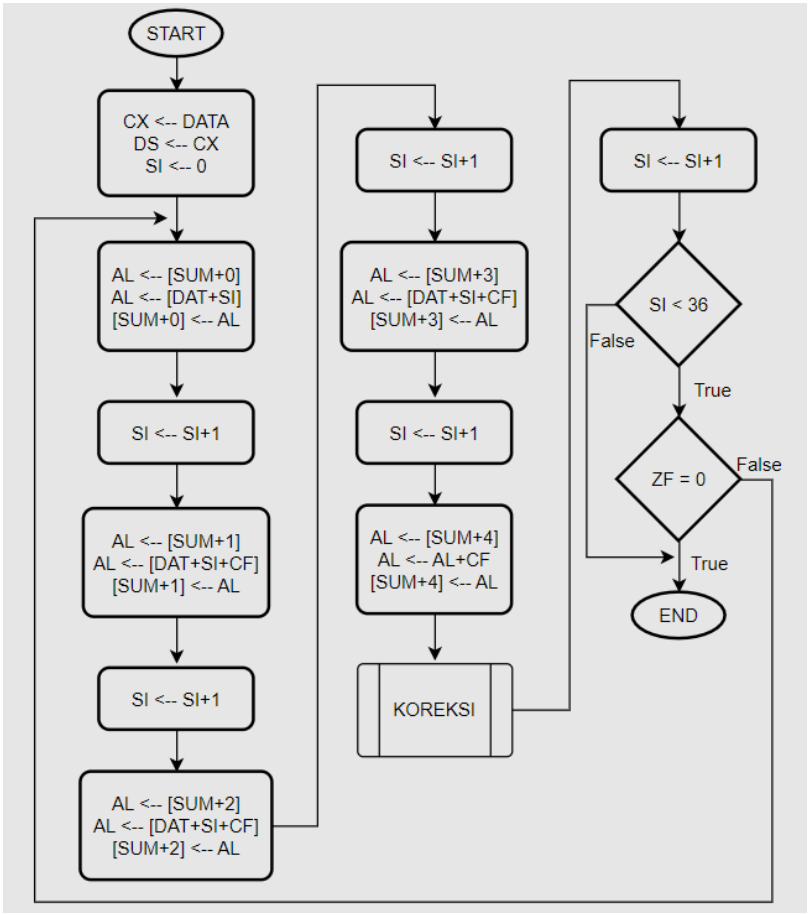
L6: MOV [BM3+SI],DX
    MOV [BM3+SI+2],CX

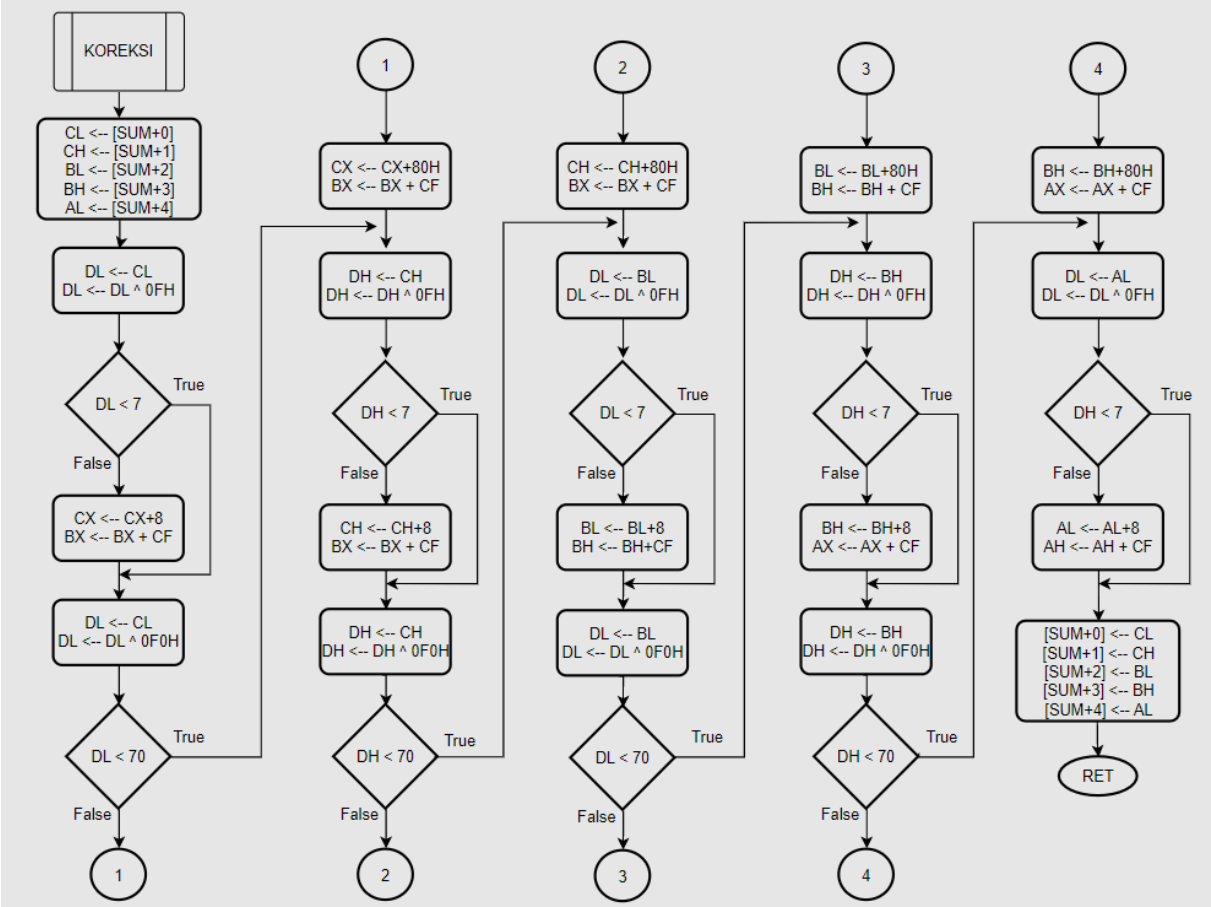
    RET
    KOREKSI ENDP
END MAIN
```

Output



5. Flowchart





Program

```
TITLE PENJ 9 DATA OKTAL
.MODEL SMALL
.STACK 100H
.DATA

;7654324+6767543+7766554+7454326+6426765+7755643+
;7776665+5556667+7747576 = 102441373o

;PROGRAMMER : ELSADANIKA MAURA SITORUS
;NIM        : 211344013
;TANGGAL    : 08/02/23

DAT DB 24H,43H,65H,7,43H,75H,76H,6
    DB 54H,65H,76H,7,26H,43H,45H,7
    DB 65H,67H,42H,6,43H,56H,75H,7
    DB 65H,66H,77H,7,67H,66H,55H,5
    DB 76H,75H,74H,7
SUM DB 5 DUP(0)

.CODE
MAIN PROC
    MOV CX,@DATA
    MOV DS,CX
```

```
    MOV SI,0

AWAL:MOV AL,[SUM+0]
    ADD AL,[DAT+SI]
    MOV [SUM+0],AL
    INC SI

    MOV AL,[SUM+1]
    ADC AL,[DAT+SI]
    MOV [SUM+1],AL
    INC SI

    MOV AL,[SUM+2]
    ADC AL,[DAT+SI]
    MOV [SUM+2],AL
    INC SI

    MOV AL,[SUM+3]
    ADC AL,[DAT+SI]
    MOV [SUM+3],AL

    MOV AL,[SUM+4]
    ADC AL,0
    MOV [SUM+4],AL

    CALL KOREKSI
    INC SI
    CMP SI,36
    JNZ AWAL


    MOV AH,4CH
    INT 21H
    MAIN ENDP

KOREKSI PROC
        MOV CL,[SUM+0]
        MOV CH,[SUM+1]
        MOV BL,[SUM+2]
        MOV BH,[SUM+3]
        MOV AL,[SUM+4]

        MOV DL,CL
        AND DL,0FH
        CMP DL,7
        JBE L1
        ADD CX,8    ;BARU NGOREKSI 1KOTAK DI DL
        ADC BX,0    ;SUPAYA CARRY GA MEREMBET SAMPAI AKHIR

L1:     MOV DL,CL
        AND DL,0F0H
        CMP DL,70H
        JBE L2
        ADD CX,80H
        ADC BX,0

L2:     MOV DH,CH
        AND DH,0FH
        CMP DH,7
```

```
        JBE L3
        ADD CH,8
        ADC BX,0

L3:     MOV DH,CH
        AND DH,0F0H
        CMP DH,70H
        JBE L4
        ADD CH,80H
        ADC BX,0

L4:     MOV DL,BL
        AND DL,0FH
        CMP DL,7
        JBE L5
        ADD BL,8
        ADC BH,0

L5:     MOV DL,BL
        AND DL,0F0H
        CMP DL,70H
        JBE L6
        ADD BL,80H
        ADC BH,0

L6:     MOV DH,BH
        AND DH,0FH
        CMP DH,7
        JBE L7
        ADD BH,8
        ADC AX,0

L7:     MOV DH,BH
        AND DH,0F0H
        CMP DH,70H
        JBE L8
        ADD BH,80H
        ADC AX,0

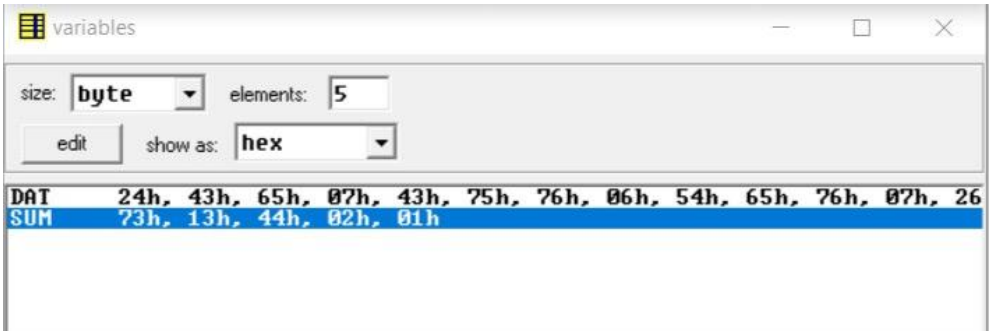
L8:     MOV DL,AL
        AND DL,0FH
        CMP DL,7
        JBE L9
        ADD AL,8
        ADC AH,0

L9:     MOV [SUM+0],CL
        MOV [SUM+1],CH
        MOV [SUM+2],BL
        MOV [SUM+3],BH
        MOV [SUM+4],AL

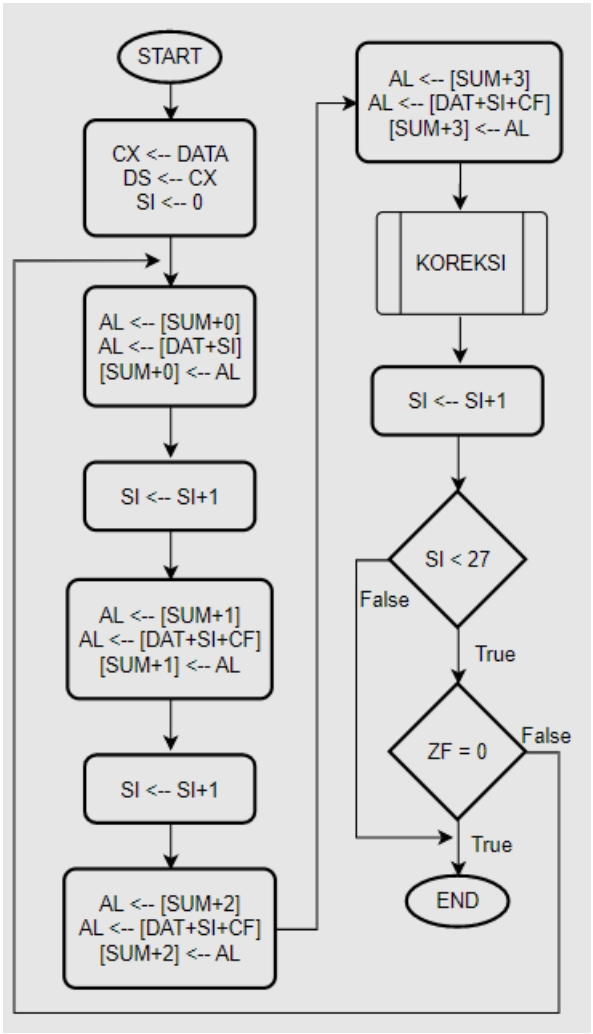

        RET    ;SETIAP PROGRAM PROCEDUR SUBROUTING DIAKHIRI DG RET
    KOREKSI ENDP

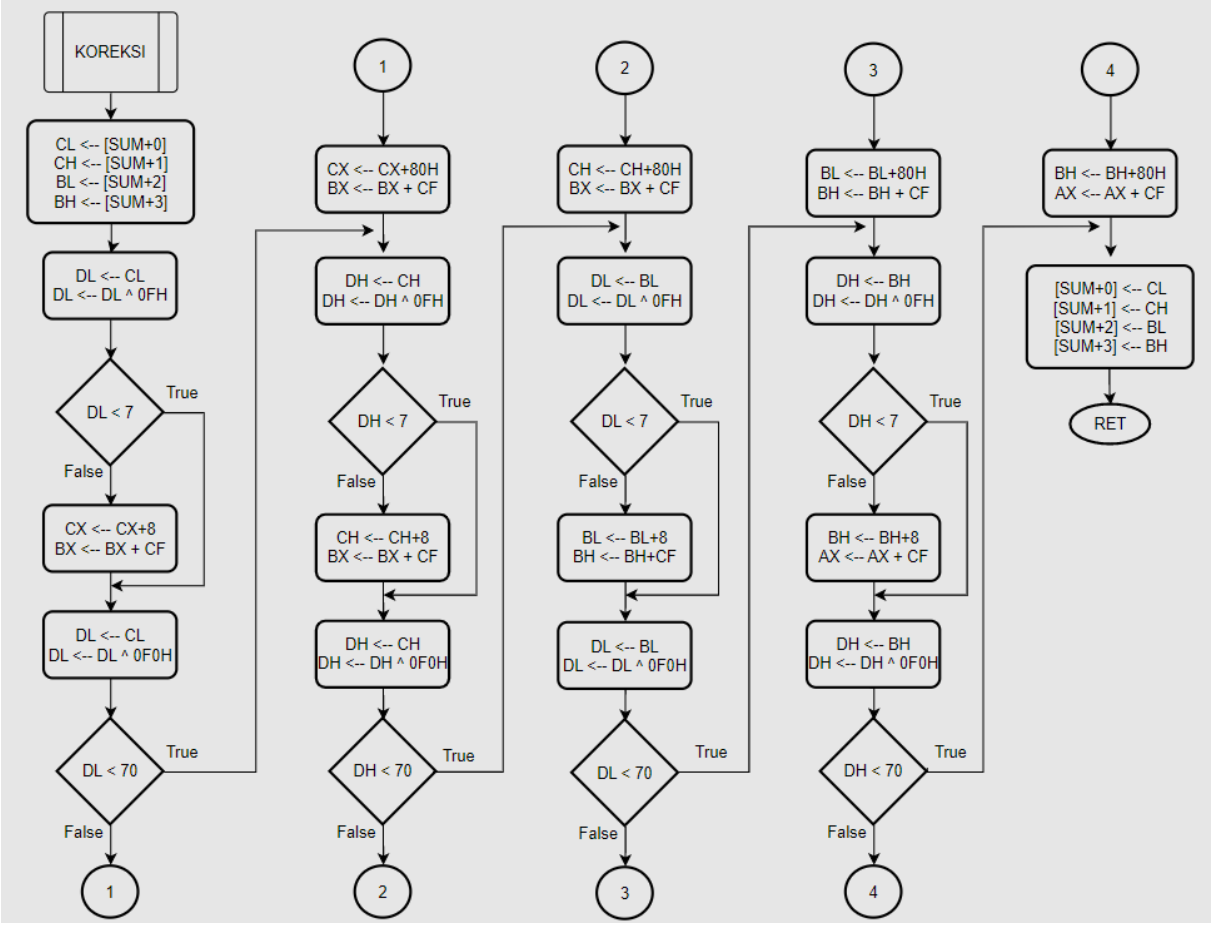
END MAIN
```

Output



6. Flowchart





Program

```
TITLE PENJ 9 DATA OKTAL 5 DIGIT
.MODEL SMALL
.STACK 100H
.DATA

;74324+67543+77554+74326+64265+77643+76765+67677+74743 = 01030150

DAT DB 24H,43H,7,43H,75H,6
    DB 54H,75H,7,26H,43H,7
    DB 65H,42H,6,43H,76H,7
    DB 65H,67H,7,77H,76H,6
    DB 43H,47H,7
SUM DB 4 DUP(0)

.CODE
MAIN PROC
     MOV CX,@DATA
     MOV DS,CX
     MOV SI,0

AWAL:MOV AL,[SUM+0]
     ADD AL,[DAT+SI]   ;JMLHKN
     MOV [SUM+0],AL    ;SIMPAN
     INC SI
```

```
        MOV AL,[SUM+1]
        ADC AL,[DAT+SI]
        MOV [SUM+1],AL
        INC SI

        MOV AL,[SUM+2]
        ADC AL,[DAT+SI]
        MOV [SUM+2],AL


        MOV AL,[SUM+3]
        ADC AL,0
        MOV [SUM+3],AL

        CALL KOREKSI
        INC SI
        CMP SI,27
        JNZ AWAL

        MOV AH,4CH
        INT 21H
        MAIN ENDP

KOREKSI PROC

           MOV CL,[SUM+0]
           MOV CH,[SUM+1]
           MOV BL,[SUM+2]
           MOV BH,[SUM+3]

           MOV DL,CL
           AND DL,0FH
           CMP DL,7
           JBE L1
           ADD CX,8     ;BARU NGOREKSI 1KOTAK DI DL
           ADC BX,0     ;SUPAYA CARRY GA MEREMBET SAMPAI AKHIR

L1:        MOV DL,CL
           AND DL,0F0H
           CMP DL,70H
           JBE L2
           ADD CX,80H
           ADC BX,0

L2:        MOV DH,CH
           AND DH,0FH
           CMP DH,7
           JBE L3
           ADD CH,8
           ADC BX,0

L3:        MOV DH,CH
           AND DH,0F0H
           CMP DH,70H
           JBE L4
           ADD CH,80H
           ADC BX,0

L4:        MOV DL,BL
```

```
        AND DL,0FH
        CMP DL,7
        JBE L5
        ADD BL,8
        ADC BH,0

L5:     MOV DL,BL
        AND DL,0F0H
        CMP DL,70H
        JBE L6
        ADD BL,80H
        ADC BH,0

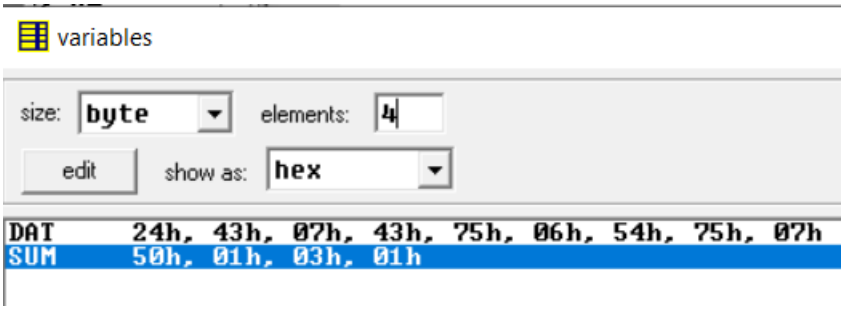
L6:     MOV DH,BH
        AND DH,0FH
        CMP DH,7
        JBE L7
        ADD BH,8

L7:     MOV [SUM+0],CL
        MOV [SUM+1],CH
        MOV [SUM+2],BL
        MOV [SUM+3],BH


        RET    ;SETIAP PROGRAM PROCEDUR SUBROUTING DIAKHIRI DG RET
    KOREKSI ENDP

END MAIN
```

Output



F. Analisis Data

Praktikum penjumlahan data-data bilangan oktal pada intel 8086 menggunakan simulator EMU8086 yang telah dilakukan menghasilkan data yang sesuai antara perhitungan teori/manual dengan perhitungan berdasarkan program yang telah dibuat. Berikut adalah analisis data setiap program yang dibuat yang dibagi dalam urutan nomor soal:

1. **Menjumlahkan 10 pasang data oktal 2 digit yang berada pada BM1 dan BM2. Simpan hasil penjumlahan pada BM3.**

   Data yang akan dijumlahkan dideklarasikan pada memory area BM1 dan BM2 dalam bentuk 8 bit dengan penggunaan Define Byte. BM3 digunakan untuk menyimpan hasil penjumlahan. Pada program pertama ini digunakan perintah MOV CX,@DATA ; MOV DA,CX ; dan MOV SI,0 sebagai langkah awal mengatur register untuk menginputkan data-data pada deklarasi variabel awal. Program utama dimulai dengan label AWAL: MOV DL,[BM1+SI] hal ini menandakan bahwa data pada BM1 akan dimasukkan bertahap ke register DL mulai dari SI = 0 hingga SI = 9. Selanjutnya data pada register DL dijumlahkan dengan BM2 dengan perintah ADD DL,[BM2+SI]. Data pada register DL kembali dipindahkan ke memory stack BM3 setelah penjumlahan tersebut secara bertahap mengikuti penambahan nilai SI.

   Ketika data telah selesai dijumlahkan, perlu dilakukan koreksi. Hal ini dikarenakan prosesor intel hanya mengenal bilangan heksadesimal, sehingga data perlu dikoreksi jika diinginkan hasil output bilangan oktal. Proses koreksi dilakukan dengan menggunakan procedure baru, maka pada main procedure digunakan perintah CALL KOREKSI untuk memanggil fungsi koreksi. Setelah proses koreksi dijalankan, main proc melanjutkan ke INC SI dan CMP SI,10 serta JNZ AWAL. Ketika nilai SI belum mencapai 10 maka program akan melompat ke label AWAL: namun ketika SI sudah mencapai 10 maka program akan berhenti dikarenakan Zero Flag akan bernilai 0. Pada saat ZF = 0 maka program akan berhenti, dan proses penjumlahan data oktal telah selesai dijalankan.

   Proses koreksi pada program pertama ini dilakukan sebanyak dua kali, yaitu hanya pada satu register dengan memisahkan satuan dengan delapanannya. Hal tersebut dilakukan karena penjumlahan data 2 digit akan menghasilkan output dalam 1 stack memory saja. Koreksi dimulai dari pemindahkan data dari BM3 ke register AH dengan perintah MOV AH,[BM3+SI]. Dilanjutkan dengan memindahkan data AL ke BL. Setelah itu, data pada register BL di-AND-kan dengan 0FH. Proses AND ini dilakukan untuk memisahkan satuan dari delapanan. Setelah itu diunakan CMP BL,7. Jika nilainya lebih besar dari 7 maka akan data satuan dijumlahkan dengan 8. Jika nilainya tidak lebih besar atau kurang dari dari 7 maka akan dilanjutkan ke koreksi data delapanan dengan perintah JBE L1. Digunakan juga perintah ADC AH,0 supaya carry yang ada tetap diperhitungkan.

   Koreksi untuk delapanan memiliki tahap yang similar dengan satuan. Yaitu dengan memindahkan data pada register AL ke BL. Setelah itu data pada register BL di-AND-kan dengan 0F0H dan dilakukan CMP BL,70H. Jika nilainya lebih

besar dari 70H maka dijumlahkan dengan 80H. Jika nilai tidak lebih besar atau kurang dari 70H maka dilanjutkan ke perintah label L2 dengan instruksi JBE L2. Setelah itu, ADC AH,0 sehingga carry yang ada dimasukkan ke register AH. Pada label L2 digunakan perintah MOV[BM3+SI],AH sehingga data yang sudah dikoreksi dikembalikan ke memory stack BM3. Pada procedure KOREKSI digunakan perintah RET atau return untuk mengembalikan nilai pada fungsi koreksi ke fungsi main atau main procedure.

Hasil akhir data ditunjukkan pada variables dengan keterangan BM3. Sesuai dengan perhitungan, didapatkan hasil 172o, 300o, 302o, 265o, 205o, 153o, 152o, 171o, 222o, 347o. Dengan demikian, program ini dapat dinyatakan berhasil dan sesuai dengan keinginan pada soal.

2. **Menjumlahkan 10 pasang data oktal 4 digit yang berada pada BM1 dan BM2. Simpan hasil penjumlahan pada BM3 dan BM4.**

Program kedua ini menggunakan konsep dan bentuk yang sama dengan program kesatu pada main procedure nya. Perbedaan yang terdapat adalah pada penggunaan Define Word dikarenakan data 4 digit bernilai 16 bit. Selain itu digunakan register 16 bit yaitu DX untuk tempat menjumlahkan data sebelum dikoreksi. Setelah perintah CALL KOREKSI digunakan dua kali increment SI dan sekali increment DI. Register segment SI digunakan untuk kenaikan nomor data dari data ke-0 hingga data ke-9. Sedangkan register segment DI digunakna untuk kenaikan nomor hasil carry setelah dikoreksi pada procedure koreksi.

Proses koreksi pada program kedua ini dilakukan sebanyak lima kali, yaitu pada satu register 16 bit dan satu register 8 bit dengan memisahkan satuan dengan delapanannya pada masing-masing stack memory. Koreksi dilakukan pada register DL sebanyak 2 kali, register DH sebanyak 2 kali, dan register AL sebanyak 1 kali. Koreksi ke-lima yang menggunakan register AL adalah untuk memasukkan nilai carry dengan penggunaan BM4 sebagai memory stacknya dan register segment DI sebagai penanda nilai datanya. Sedangkan koreksi ke-satu hingga ke-empat yang menggunakan register DX adalah untuk memasukkan nilai hasil penjumlahan dengan penggunaan BM3 ke memory stacknya dan register segment SI sebagai penanda nilai datanya.

Masih pada procedure koreksi, ketika semua data telah selesai dikoreksi, maka data pada register DX dipindahkan ke memory stack BM3. Sedangkan data pada register AL dipindahkan ke BM4. Pada procedure KOREKSI digunakan perintah RET atau return untuk mengembalikan nilai pada fungsi koreksi ke fungsi main atau main procedure. Hasil akhir data ditunjukkan pada variables dengan keterangan BM3 dan BM4. Sesuai dengan perhitungan, didapatkan hasil pada BM3 = 5322h, 3715h, 2034h, 0016h, 2275h, 7405h, 5602h, 3623h, 0027h, 6006h dan pada BM4 = 00h, 01h, 01h, 01h, 01h, 00h, 00h, 01h, 01h, 01h . Dengan demikian, program ini dapat dinyatakan berhasil dan sesuai dengan keinginan pada soal.

3. **Menjumlahkan data oktal 7 digit yang berada pada BM1 dan BM2. Simpan hasil penjumlahan pada BM3.**

Pada program ketiga, digunakan deklarasi memory area BM1 dan BM2 dengan tipe DW. Data-data bernilai 7 digit dibagi menjadi 2 yaitu 3 digit high byte dan 4 digit low byte. Hasil penjumlahan data-data disimpan pada BM3 dengan jumlah 10 dikarenakan data 7 digit berjumlah 5 data, sehingga diperlukan 10 ruang pada memory stack jika dalam bentuk define word.

Proses yang terjadi pada program 3 adalah sama dengan program 1. Hak yang membedakan adalah increment dilakukan sebanyak dua kali, dan compare CMP SI, 20 dikarenakan jumlah keseluruhan data adalah 20. Proses koreksi pada program ketiga ini dilakukan sebanyak empat kali, yaitu pada dua buah register 16 bit dengan memisahkan satuan dengan delapanannya pada masing-masing stack memory. Koreksi dilakukan pada register DL sebanyak 2 kali, register DH sebanyak 2 kali, dan nilai carry dimasukkan ketika proses koreksi pada register DH ke register CX. Koreksi ke-satu hingga ke-empat yang menggunakan register DX adalah untuk memasukkan nilai hasil penjumlahan dengan penggunaan BM3 ke memory stacknya dan register segment SI sebagai penanda nilai datanya, nilai carry pada register CX dimasukkan ke BM3 dengan perumusan BM3+SI+2.

Hasil akhir data ditunjukkan pada variables dengan keterangan BM3. Sesuai dengan perhitungan, didapatkan hasil pada BM3 = 15640245h, 11663071h, 15443313h, 11611554h, 10431752h. Dengan demikian, program ini dapat dinyatakan berhasil dan sesuai dengan keinginan pada soal.

4. **Menjumlahkan 4 pasang data oktal 11 digit. Data input dan outputnya tentukan sendiri.**

Pada program keempat, digunakan deklarasi memory area BM1 dan BM2 dengan tipe DW. Data-data bernilai 11 digit dibagi menjadi 3 yaitu 3 digit high byte, 4 digit medium byte, dan 4 digit low byte. Hasil penjumlahan data-data disimpan pada BM3 dengan jumlah 12 dikarenakan data 11 digit berjumlah 4 pasang data, sehingga diperlukan 12 ruang pada memory stack jika dalam bentuk define word.

Proses yang terjadi pada program 4 adalah sama dengan program 1 dan 3. Hal yang membedakan adalah increment dilakukan sebanyak dua kali, dan compare CMP SI, 24 dikarenakan jumlah keseluruhan data adalah 24 buah. Proses koreksi pada program keempat ini dilakukan sebanyak enam kali, yaitu pada dua buah register 16 bit dengan memisahkan satuan dengan delapanannya pada masing-masing stack memory. Koreksi dilakukan pada register DL sebanyak 2 kali, register DH sebanyak 2 kali, dan regiter CL sebanyak 2 kali. Koreksi ke-satu hingga ke-enam yang menggunakan register DX dan CX adalah untuk memasukkan nilai hasil penjumlahan dengan penggunaan BM3 ke memory stacknya dan register segment SI sebagai penanda nilai datanya, nilai carry pada register CX dimasukkan ke BM3 dengan perumusan BM3+SI+2.

Data yang digunakan adalah sebagai berikut:

BM1 DW 3344H,2223H,0111H,3344H

```
    DW 2223H,0111H,3344H,2223H
    DW 0111H,3344H,2223H,0111H
BM2 DW 3344H,2223H,0111H,3344H
    DW 2223H,0111H,3344H,2223H
    DW 0111H,3344H,2223H,0111H
```

Data tersebut adalah 11122233344 yang di-duplicate sebanyak 4 kali. Dari data-data tersebut dihasilkan nilai 2224446671o yang berjumlah 4 buah. Hasil akhir data ditunjukkan pada variable dengan keterangan BM3. Hasil ini sesuai dengan perhitungan teori/manual menggunakan perumusan dan calculator. Output dapat dilihat pada bagian hasil praktikum. Dengan demikian, program ini dapat dinyatakan berhasil dan sesuai dengan keinginan pada soal.

5. **Menjumlahkan 9 data oktal berikut: 7654324 + 6767543 + 7766554 + 7454326 + 6426765 + 7755643 + 7776665 + 5556667 + 7747576. Simpan data input dan output pada data area.**

Program kelima memiliki bentuk yang berbeda dari program-program sebelumnya. Dalam satu memory stack terdapat 9 pasang data oktal. Data dideklarasikan dengan DAT DB sehingga setiap data diubah ke bentuk byte. Data menjadi berjumlah 36 buah. Hasil penjumlahannya akan disimpan ke memory stack SUM DB 5 DUP(0). Menjumlahkan data oktal dengan jumlah banyak akan menggunakan mempry stack yang disesuaikan dengan kebutuhan. Untuk menyesuaikannya, digunakan perhitungan seperti berikut.

Penjumlahan 9 data oktal 7 digit	
[SUM+4][SUM+3][SUM+2][SUM+1][SUM+0] 00 [DAT+3][DAT+2][DAT+1][DAT+0] ---------- + [SUM+4][SUM+3][SUM+2][SUM+1][SUM+0] AL BH BL CH CL	→ data yang dijumlahkan → data akhir disimpan → untuk proses koreksi
Hasil = 16 bit	

Koreksi dilakukan untuk setiap kali proses penjumlahan data oktal. Pada Main Proc digunakan perintah MOV CX,@DATA ; MOV DA,CX ; dan MOV SI,0 sebagai langkah awal mengatur register untuk menginputkan data-data pada deklarasi variabel awal. Program utama dimulai dengan label AWAL: MOV AL,[SUM+0] hal ini menandakan bahwa data hasil penjumlahan yang ada di [SUM+0] akan dimasukkan ke register AL. Setelah itu, dilanjutkan dengan perintah ADD AL,[DAT+SI] sehingga register AL akan dijumlahkan dengan data ke-0 yang ada di memory area. Jika sudah selesai dijumlahkan, data dikembalikan ke memory stack [SUM+0] dan dilanjutkan dengan INC SI. Sehingga saat ini nomor data adalah 1.

Tahap selanjutnya adalah MOV AL,[SUM+1] lalu menjumlahkan data ke 1 dengan carry-nya atau ADC AL,[DAT+SI] dengan [SUM+1] pada register AL. Ketika selesai dijumlahkan, data dikembalikan ke memory stack [SUM+1] dan

dilakukan increment SI, sehingga saat ini nomor data adalah 2. Jika tahap dua ini telah selesai dilanjutkan ke tahap ketiga yaitu MOV AL,[SUM+2] lalu menjumlahkan data ke 2 dengan carry-nya atau ADC AL,[DAT+SI] dengan [SUM+2] pada register AL. Ketika selesai dijumlahkan, data dikembalikan ke memory stack [SUM+2] dan dilakukan increment SI, sehingga saat ini nomor data adalah 3. Hal tersebut dilakukan hingga [SUM+4] dengan perbedaan pada [SUM+3] dan [SUM+4] tidak lagi dilakukan proses increment dan pada [SUM+4] data dijumlahkan dengan carrynya saja yaitu ADC AL,0.

Ketika tahapan ini telah selesai dijalankan, artinya 1 pasang data oktal telah semua masuk ke memory stack. Dipanggil fungsi koreksi dan proses koreksi dilakukan. Jika proses koreksi telah selesai dilakukan maka nilai SI dinaikkan sebanyak 1 angka dan dilakukan CMP SI,36 serta JNZ AWAL. Proses pertama nilai SI adalah 3, sehingga dilanjutkan dengan penjumlahan pada label AWAL: dengan data selanjutnya. Hal ini terus berulang hingga data ke-9 selesai dijumlahkan.

Proses koreksi pada program ke-lima ini dilakukan sebanyak sembilan kali, yaitu pada dua buah register 16 bit dan satu buah register 8 bit dengan memisahkan satuan dengan delapanannya pada masing-masing stack memory. Koreksi dilakukan pada register CL sebanyak 2 kali, register CH sebanyak 2 kali, nilai carry dimasukkan ke register BX. Lalu pada register BL sebanyak 2 kali dengan carrynya dimasukkan ke register BH, register BH sebanyak 2 kali dengan nilai carrynya dimasukkan ke register AX. Yang terakhir adalah register AL sebanyak 1 kali dengan nilai carry nya dimasukkan ke register AH. Setelah koreksi di register-register tersebut, data dikembalikan ke memory stack [SUM+0], [SUM+1], [SUM+2], [SUM+3], dan [SUM+4] sesuai registernya masing-masing yang dijelaskan pada tabel di awal pembahasan soal 5 ini. Setiap program procedure subrouting diakhiri dengan RET.

Dari data-data yang dijumlahkan tersebut, didapatkan hasil 102441373o. Hasil ini sesuai dengan perhitungan teori manual maupun menggunakan kalkulator. Dengan demikian, program ini dapat dinyatakan berhasil dan sesuai dengan keinginan pada soal.

6. **Menjumlahkan 9 data oktal berikut: 74324 + 67543 + 77554 + 74326 + 64265 + 77643 + 76765 + 67677 + 74743. Simpan data input dan output pada data area.**
Program ke-enam ini menggunakan memory area dengan deklarasi variabel DAT DB sehingga setiap data diubah bentuknya ke byte. Data menjadi berjumlah 27 buah. Hasil penjumlahannya akan disimpan ke memory stack SUM DB 4 DUP(0). Main Proc pada program ini memiliki kesamaan dengan program ke-lima. Perbedaan yang terdapat adalah jumlah SUM hanya mencapai [SUM+3]. Hal tersebut dinyatakan pada tabel berikut.

Penjumlahan 9 data oktal 5 digit
[SUM+3][SUM+2][SUM+1][SUM+0] 01 [DAT+2][DAT+1][DAT+0] → data yang dijumlahkan ---------- +

[SUM+3][SUM+2][SUM+1][SUM+0] BH BL CH CL	→ tempat data akhir disimpan → untuk proses koreksi
Hasil = 16 bit	

Proses koreksi dilakukan sebanyak 7 kali pada dua buah register 16 bit yaitu CX dan BX dengan memisahkan satuan dengan delapanannya pada masing-masing stack memory. Koreksi dilakukan pada register CL sebanyak 2 kali, register CH sebanyak 2 kali, nilai carry dimasukkan ke register BX. Lalu pada register BL sebanyak 2 kali dengan carrynya dimasukkan ke register BH, register BH sebanyak 1 kali. Setelah koreksi di register-register tersebut, data dikembalikan ke memory stack [SUM+0], [SUM+1], [SUM+2], dan [SUM+3] sesuai registernya masing-masing yang dijelaskan pada tabel di awal pembahasan soal 4 ini. Setiap program procedure subrouting diakhiri dengan RET.

Dari data-data yang dijumlahkan tersebut, didapatkan hasil 01030150h. Hasil ini sesuai dengan perhitungan teori manual maupun menggunakan kalkulator. Dengan demikian, program ini dapat dinyatakan berhasil dan sesuai dengan keinginan pada soal.

G. Kesimpulan

Melalui praktikum yang telah dilakukan, diketahui bahwa didalam penjumlahan bilangan oktal perlu dilakukan proses koreksi dengan memisahkan satuan dengan delapanan terlebih dahulu. Pemisahan satuan dengan delapanan ini menggunakan perintah logic AND yang dilanjutkan dengan proses compare dengan 7. Jika nialinya lebih dari 7, maka dijumlahkan dengan 8 sehingga data satuan menjadi terpisahkan. Selain itu, koreksi dapat dilakukan pada procedure yang berbeda dengan main procedure. Jika menggunakan procedure yang berbeda, maka perlu digunakan perintah RET atau return untuk setiap subrouting procedure.

Ketika data telah selesai dijumlahkan, perlu dilakukan koreksi. Hal ini dikarenakan prosesor intel hanya mengenal bilangan heksadesimal, sehingga data perlu dikoreksi jika diinginkan hasil output bilangan oktal. Proses koreksi dilakukan dengan menggunakan procedure baru, maka pada main procedure digunakan perintah CALL KOREKSI untuk memanggil fungsi koreksi.

Dalam satu register 8 bit terdiri dari delapanan dan satuan, maka dari itu proses koreksi dalam satu buah register 8 bit dilakukan sebanyak 2 kali jika data bernilai 2 digit. Hal ini berlaku kelipatan mengikuti jumlah digit pada setiap data yang dimiliki. Oleh karena itu, prosedur koreksi memiliki perintah dan instruksi yang sama namun berulang dalam jumlah yang banyak pada register yang berbeda-beda.

H. Daftar Pustaka

[1] Kho, D. Pengertian Mikroprosesor (Microprocessor) dan Cara Kerja Mikroprossesor. [teknikelektronika.com]. Available:

https://teknikelektronika.com/pengertian-mikroprosesor-microprocessor-cara-kerja-mikroprosesor-adalah/
[2] Hidayatullah, S. 2021. Pengertian Mikroprosesor Beserta Fungsi dan Sejarah Mikroprosesor. [belajaronline.net]. Available: https://www.belajaronline.net/2021/01/pengertian-mikroprosesor-fungsi-dan-sejarah-mikroprosesor.html
[3] Ardiyanto88. 2013. Perintah Dasar Bahasa Assembly Menggunakan Emulator 8086. [ardiyanto88.wordpress.com]. Available: https://ardiyanto88.wordpress.com/2013/10/25/perintah-dasar-bahasa-assembly-menggunakan-emulator-8086/